



SOFTWARE ENGINEERING: AN INTRODUCTION



What is SOFTWARE ENGINEERING?

- ▶ Software engineering is an engineering discipline that is concerned with all aspects of software production.
 - Software engineers should adopt a systematic and organised approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available.

In other words:

Software Engineering is an engineering discipline whose aim is the production of fault free software that satisfies the user's needs and that is delivered on time and within budget.

SOFTWARE CRISIS

- ▶ In the late 1960s, it became clear that the development of software is different from manufacturing other products. This is because employing more manpower (programmers) later in the software development does not always help speed up the development process. Instead, sometimes it may have negative impacts like delay in achieving the scheduled targets, degradation of software quality, etc.
- ▶ It refers to the set of problems encountered in the development of computer software during 1970s. History has many software failures such as:
 1. Year 2000(Y2K) problem
 2. In 1996, Arian-5 space rocket
 3. During the Gulf War in 1991, the United States of America used Patriot missiles as a defence against Iraqi Scud missiles.
- ▶ The major causes of software crisis were
 1. Lack of Communication between software developer and User
 2. Increase in the cost of Software Compared to Hardware
 3. Increase in the size of Software
 4. Project Management Problems
 5. Lack of adequate training in software engineering
 6. Increasing skill shortage

Some Software failures

Ariane 5

It took the European Space Agency **10 years and \$7 billion** to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit with each Launch and intended to give Europe overwhelming supremacy in the commercial space business.

The rocket was destroyed after 39 seconds of its launch, at an altitude of two and a half miles along with its payload of four expensive and uninsured scientific satellites.



Some Software Failures

The Patriot Missile

- o First time used in Gulf war.
- o Used as a defense from Iraqi Scud missiles.
 - o Failed several times including one that killed 28 US soldiers in Dhahran, Saudi Arabia.

Reasons:

A small timing error in the system's clock accumulated to the point that after 14 hours, the tracking system was no longer accurate. In the Dhahran attack, the system had been operating for more than 100 hours.



Some Software Failures

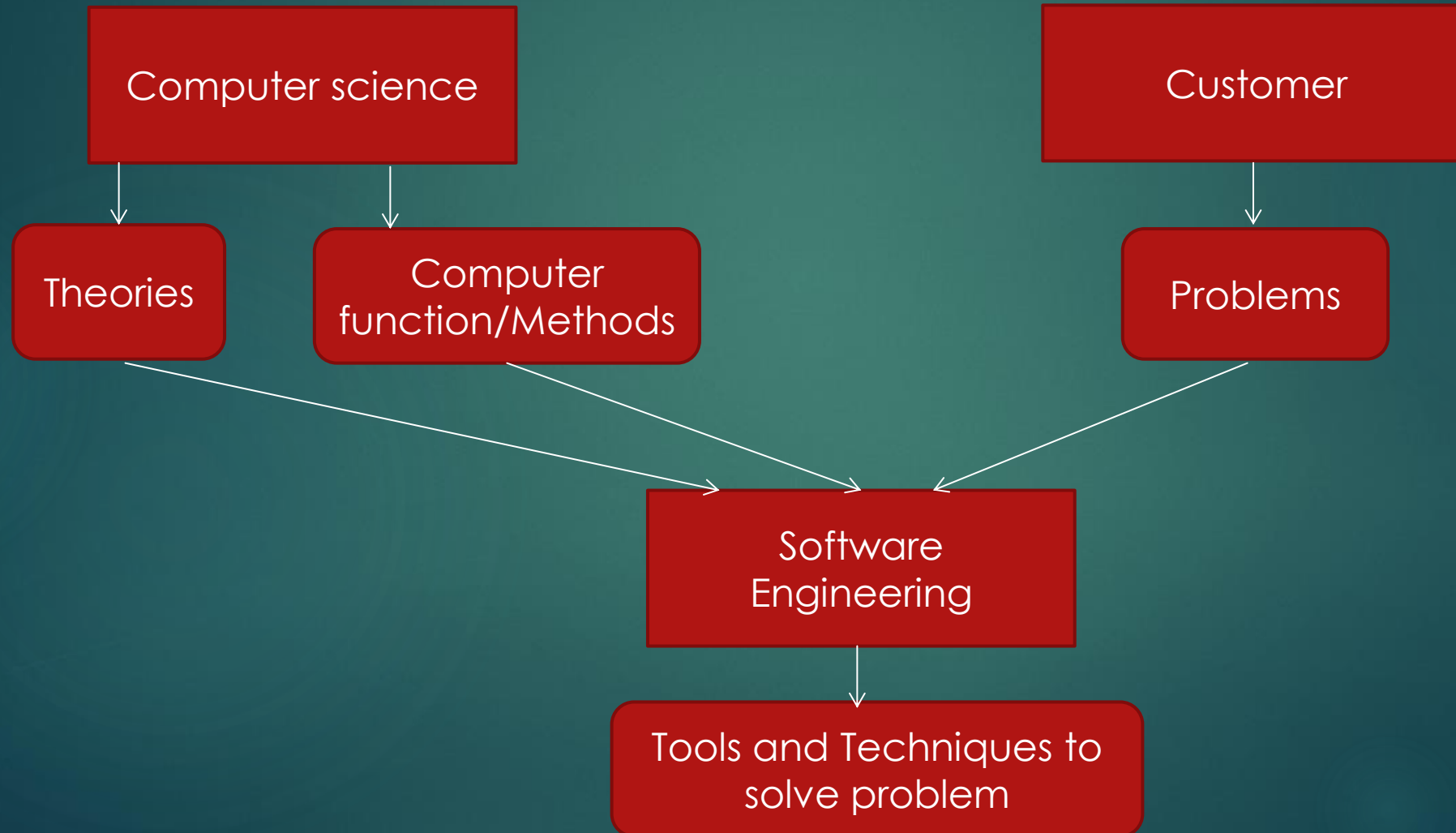
Y2K problem:

It was simply the ignorance about the adequacy or otherwise of using only last two digits of the year.

The 4-digit date format, like 1964, was shortened to 2-digit format, like 64.



SOFTWARE ENGINEERING/COMPUTER SCIENCE



Program versus Software

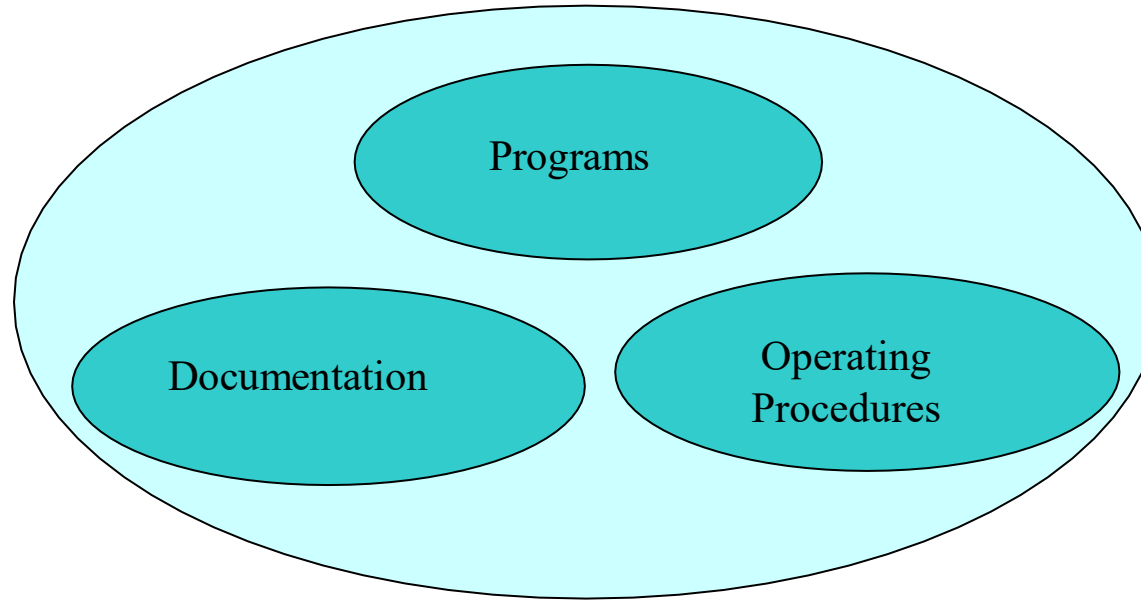


Figure: Components of software

Software=Program + Documentation + Operating Procedures

Documentation consists of different types of manuals are

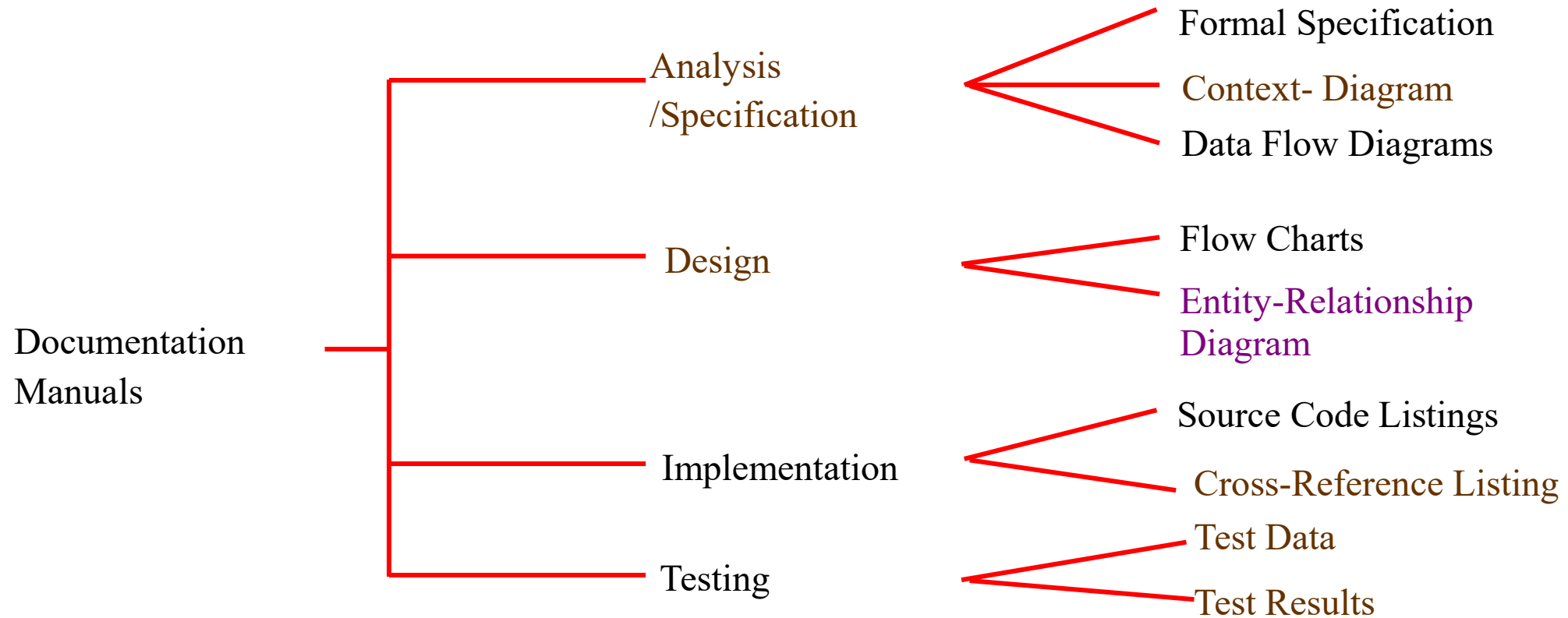


Figure: List of documentation manuals

Documentation consists of different types of manuals are

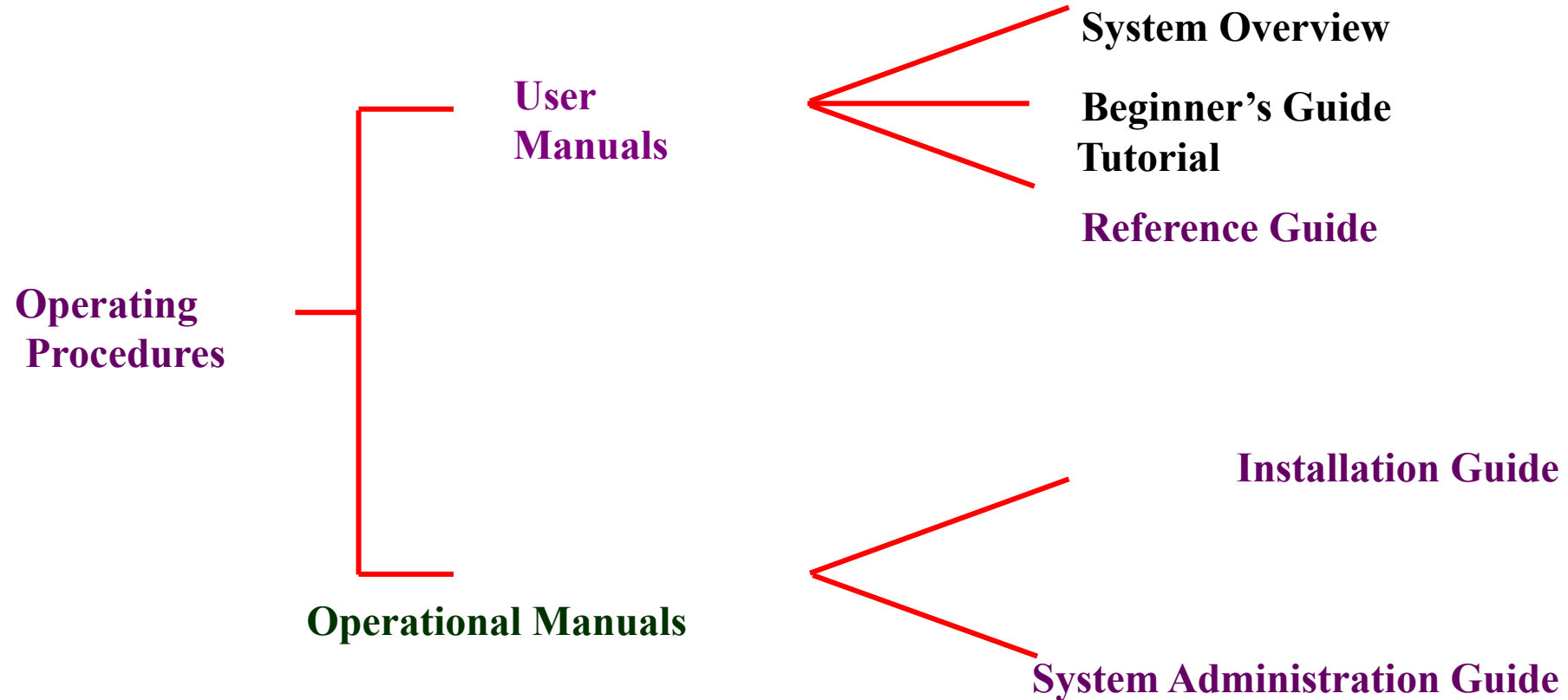


Figure: List of operating procedure manuals.

Software Characteristics

➤ Software does not wear out.

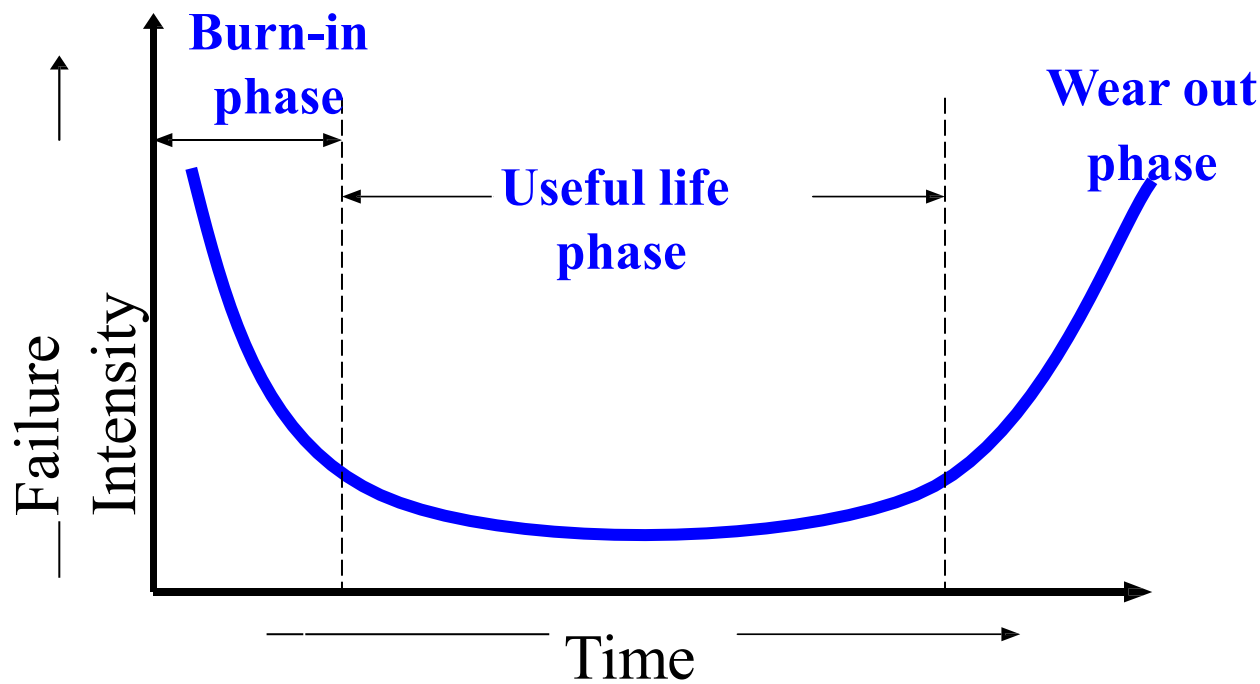


Fig: Bathtub curve for H/W products

Figure depicts failure rate as a function of time for **hardware**. The relationship, often called the "bathtub curve," indicates that hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects); defects are corrected and the failure rate drops to a steady-state level (ideally, quite low) for some period of time. As time passes, however, the failure rate rises again as hardware components suffer from the cumulative affects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply, the hardware begins to wear out.

Software becomes reliable instead of wearing out. Moreover, the Software may be retired due to Environmental changes, new requirements, new expectations etc.

Software Characteristics

- ✓ Software is developed or engineered, it is not manufactured in classical sense.
- ✓ Reusability of components.
- ✓ Software is highly flexible.
- ✓ Software is logical rather than physical system elements.

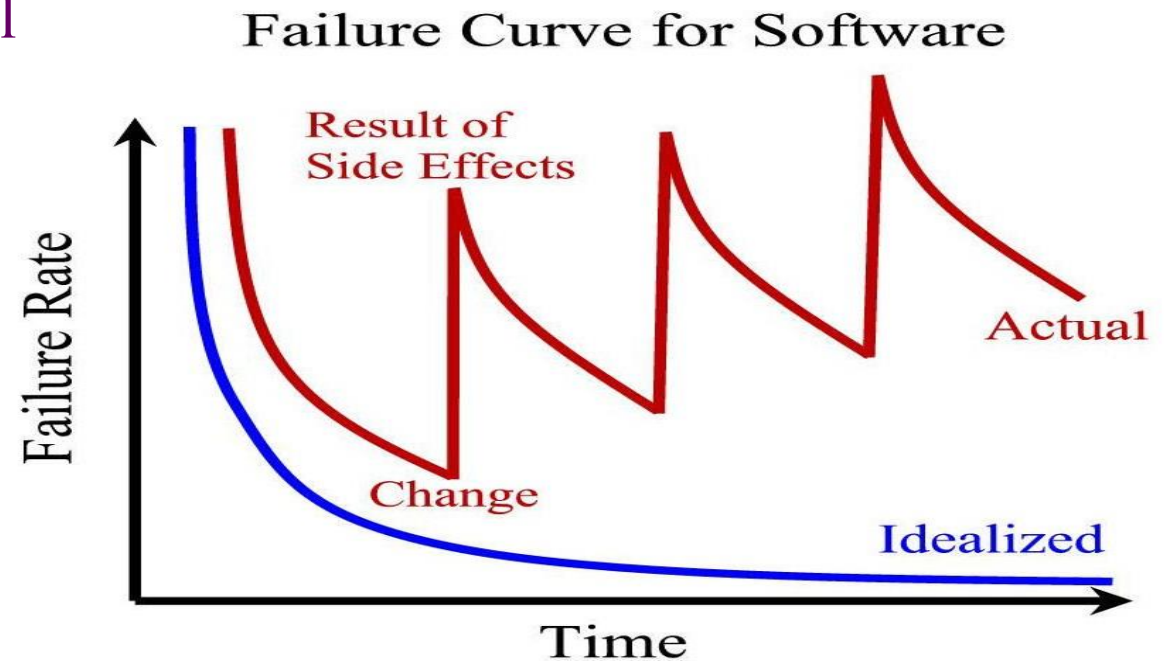
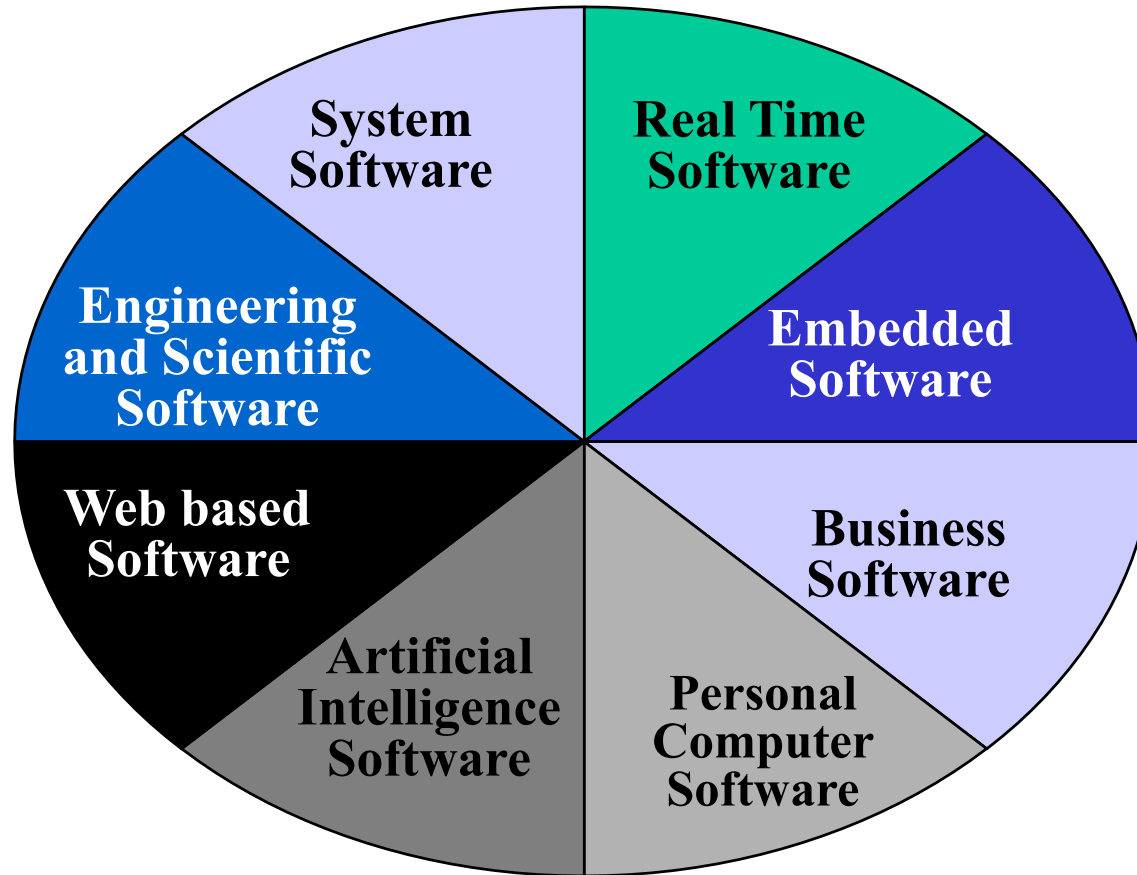


Fig: Software curve

The Changing Nature of Software/Software Applications



SOFTWARE DEVELOPMENT LIFECYCLE MODELS(SDLC MODELS)

- ▶ A software life cycle model (also called process model) is a descriptive and diagrammatic representation of the software life cycle. A life cycle model represents all the activities required to make a software product transit through its life cycle phases. In other words, a life cycle model maps the different activities performed on a software product from its inception to retirement. There are various models, and many companies adopt their own, but all have very similar patterns. Some of the models as follows.
- ▶ Water fall model or Linear Sequential or Classical waterfall Model
- ▶ Iterative waterfall model
- ▶ Evolutionary Process Model
 1. Prototyping Model
 2. Spiral Model
- ▶ Incremental Process Model
 1. Incremental Model
 2. Rapid Application Development (RAD) model
- ▶ Specialized Process Model
 1. Component Based Development Model
- ▶ V-Model
- ▶ Agile Process Models (1. Extreme Programming(XP) 2. Adaptive software development (ASD) 3. Scrum 4. Crystal)

WATERFALL MODEL



- ▶ This model was introduced by Wiston Royce in 1970.
- ▶ This model is named “waterfall model” because its diagrammatic representation resembles a cascade of waterfalls.
- ▶ In a waterfall model, each phase must be completed in its entirety before the next phase can begin.
- ▶ There are different stages to the development and the output of the first stage “flow” into the second stage and these output “flow” into the third stage and so on.
- ▶ Waterfall model is an example of plan-driven process , you must plan and schedule all of the process activities before starting work on them.
- ▶ It is also referred to as a linear sequential life cycle model or Classic life cycle model.

FEASIBILITY STUDY



REQUIREMENT ANALYSIS AND
SPECIFICATION



DESIGN



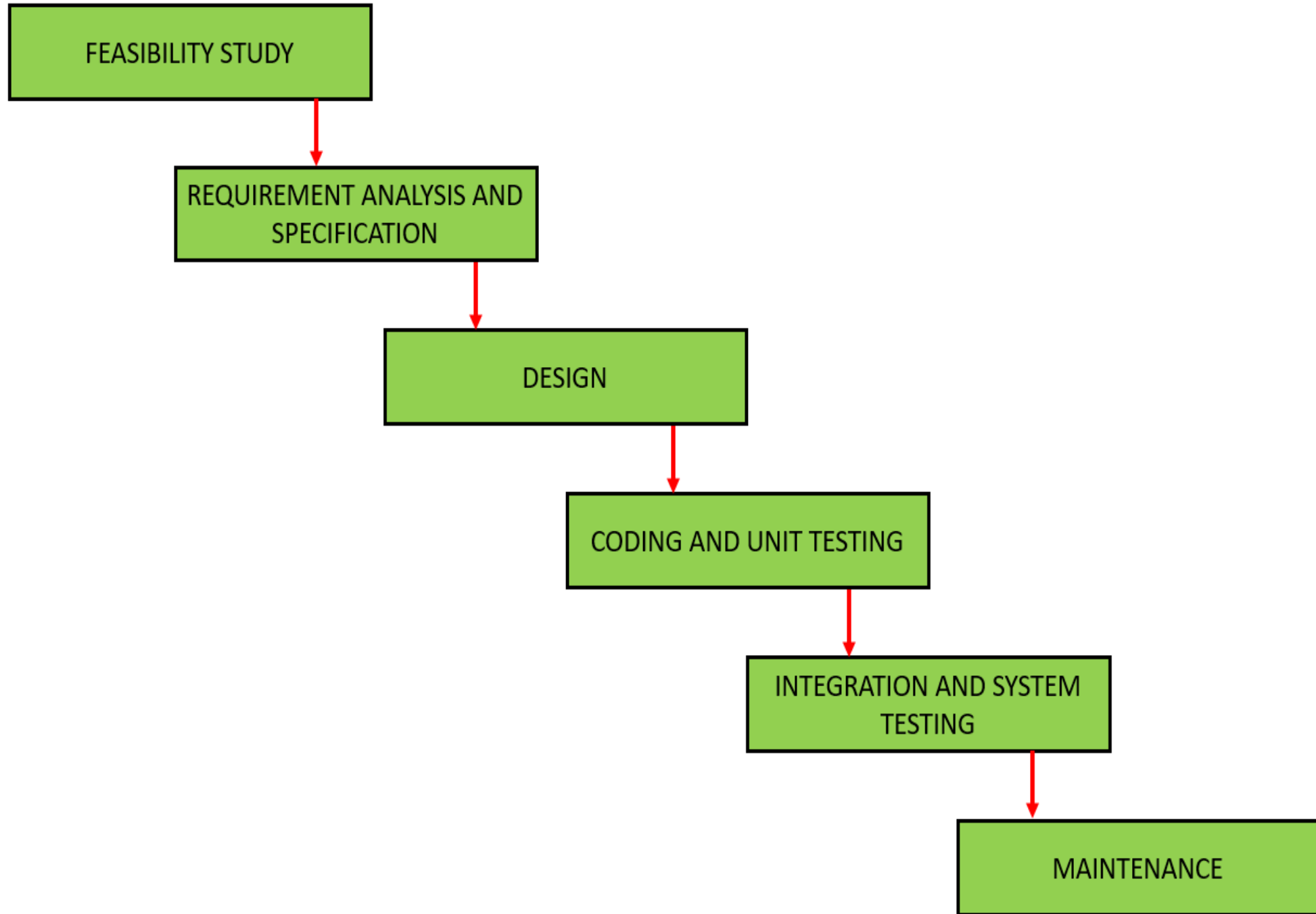
CODING AND UNIT TESTING



INTEGRATION AND SYSTEM
TESTING



MAINTENANCE



WATERFALL MODEL(contd.)

- **Feasibility Study**- The main aim of feasibility study is to determine whether it would be financially and technically feasible to develop the software.
- **Requirements analysis and specification**: The aim of the requirement analysis and specification phase is to understand the exact requirements of the customer and document them properly. This phase consists of two different activities. (i)**Requirement gathering and analysis** (ii)**Requirement specification**.
- **Design** – The requirement specifications from above phase are studied in this phase and the system design is prepared. The aim of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language.
- **Coding and Unit testing**: In coding phase software design is translated into source code using any suitable programming language. Thus each designed module is coded. The aim of the unit testing phase is to check whether each module is working properly or not.
- **Integration and System testing**: Integration of different modules are undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over a number of steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists three different kinds of testing activities as described below

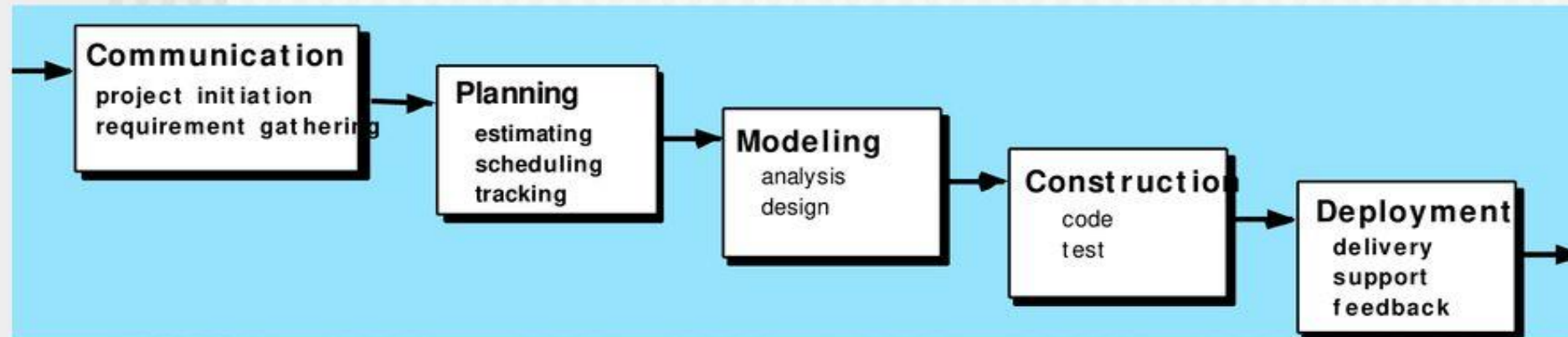
WATERFALL MODEL(contd.)

- **Alpha testing:** Alpha testing is the system testing performed by the development team.
- **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.
- **Acceptance testing:** After the software has been delivered, the customer performed the acceptance testing to determine whether to accept the delivered software or to reject it.
- ▶ **Maintenance:** Maintenance is the most important phase of a software life cycle. The effort spent on maintenance is the 60% of the total effort spent to develop a full software. There are basically four types of maintenance:
 - **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
 - **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
 - **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as work on a new computer platform or with a new operating system.
 - **Preventive Maintenance:** involves the activities to update the software in anticipation of any future problem.

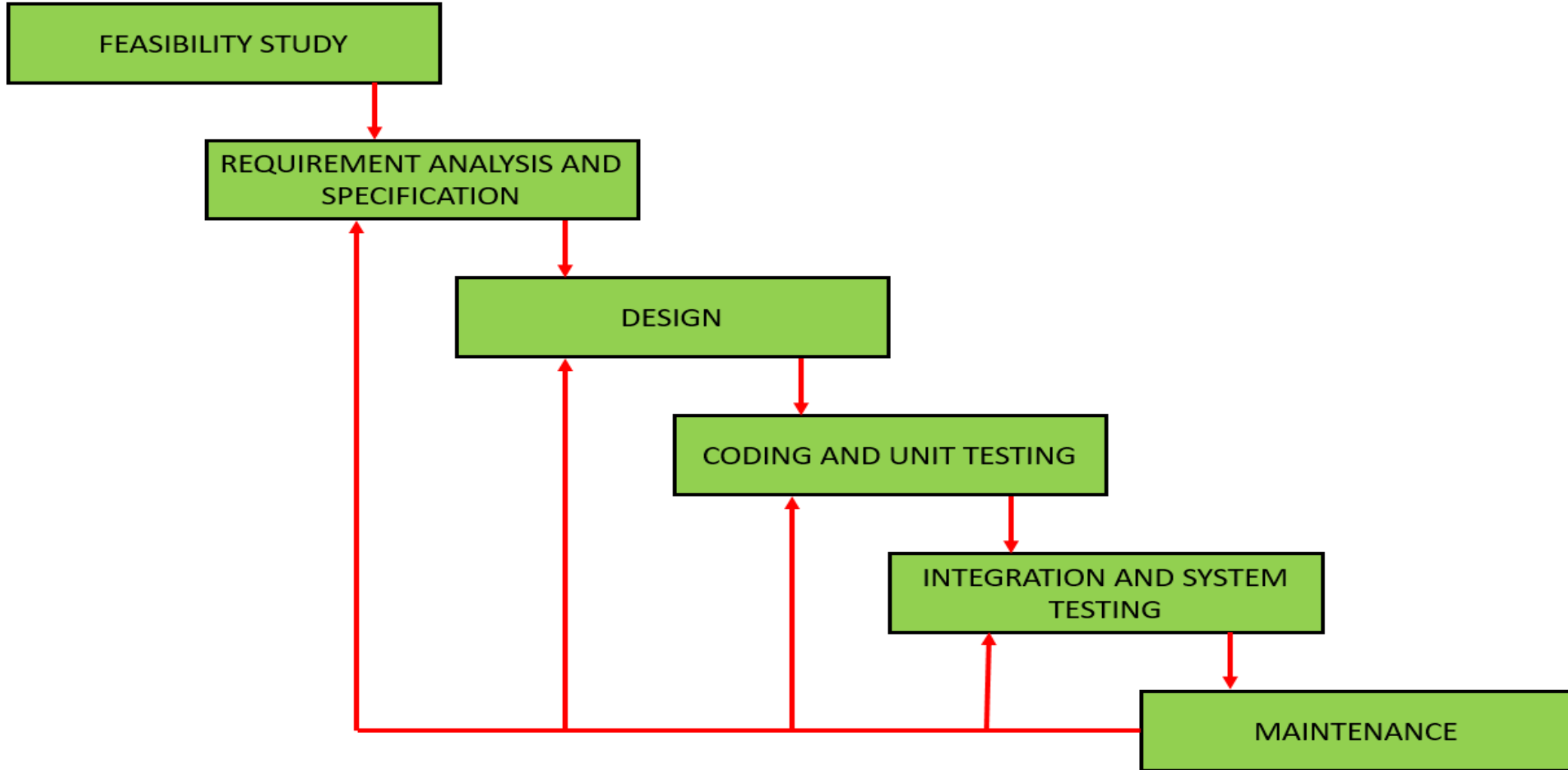
Shortcomings of the classical waterfall model

- ▶ The classical waterfall model is an idealistic one since it assumes that no development error is ever committed by the engineers during any of the life cycle phases. However, in practical development environments, the engineers do commit a large number of errors in almost every phase of the life cycle. The source of the defects can be many: oversight, wrong assumptions, use of inappropriate technology, communication gap among the project engineers, etc. These defects usually get detected much later in the life cycle. For example, a design defect might go unnoticed till we reach the coding or testing phase. Once a defect is detected, the engineers need to go back to the phase where the defect had occurred and redo some of the work done during that phase and the subsequent phases to correct the defect and its effect on the later phases. Therefore, in any practical software development work, it is not possible to strictly follow the classical waterfall model.
- ▶ It is difficult to define all requirement at the beginning of a project.
- ▶ This model is not suitable for accommodating any change.
- ▶ It does not suitable for large projects.
- ▶ A working version of the system is not seen until late in the project's life.
- ▶ It follows the “big-bang approach”, the entire software is delivered in one shot at the end. This entails heavy risks.
- ▶ It is a document driven process that requires formal documents at the end of each phase.

The Waterfall Model



Iterative Waterfall Model



Iterative Waterfall Model (Cont..)

- In a practical software development project, the classical waterfall model is hard to use. So, Iterative waterfall model can be thought of as incorporating the necessary changes to the classical waterfall model to make it usable in practical software development projects. It is almost same as the classical waterfall model except some changes are made to increase the efficiency of the software development.

The iterative waterfall model provides feedback paths from every phase to its preceding phases, which is the main difference from the classical waterfall model.

- When errors are detected at some later phase, these feedback paths allow correcting errors committed by programmers during some phase. The feedback paths allow the phase to be reworked in which errors are committed and these changes are reflected in the later phases. But, there is no feedback path to the stage – feasibility study, because once a project has been taken, does not give up the project easily.
It is good to detect errors in the same phase in which they are committed. It reduces the effort and time required to correct the errors.

Iterative Waterfall Model (Cont..)

Advantages of Iterative Waterfall Model

- ▶ **Feedback Path:** In the classical waterfall model, there are no feedback paths, so there is no mechanism for error correction. But in iterative waterfall model feedback path from one phase to its preceding phase allows correcting the errors that are committed and these changes are reflected in the later phases.
- ▶ **Simple:** Iterative waterfall model is very simple to understand and use. That's why it is one of the most widely used software development models.

Drawbacks of Iterative Waterfall Model

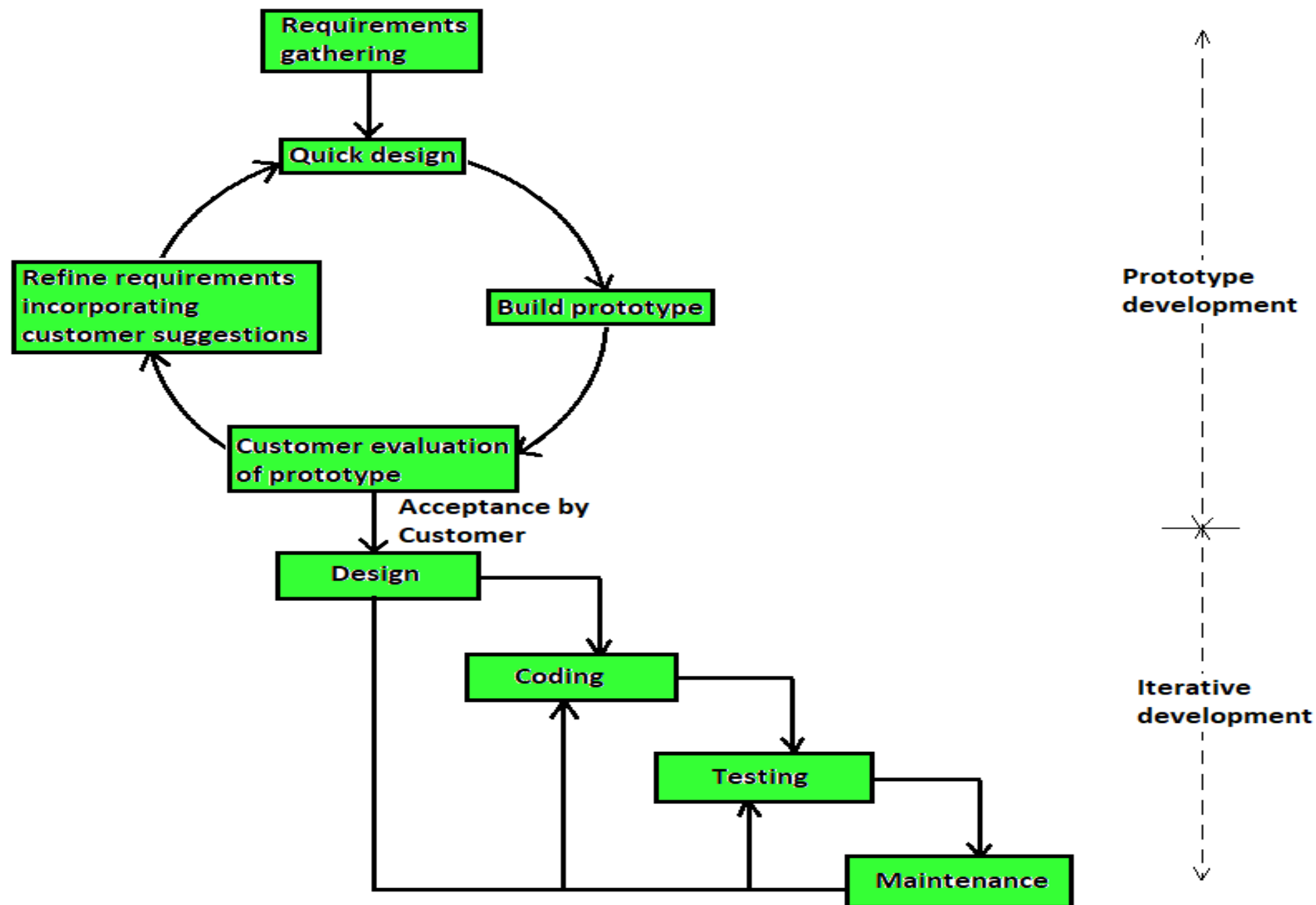
- ▶ **Difficult to incorporate change requests:** The major drawback of the iterative waterfall model is that all the requirements must be clearly stated before starting of the development phase. Customer may change requirements after some time but the iterative waterfall model does not leave any scope to incorporate change requests that are made after development phase starts.
- ▶ **Incremental delivery not supported:** In the iterative waterfall model, the full software is completely developed and tested before delivery to the customer. There is no scope for any intermediate delivery. So, customers have to wait long for getting the software.
- ▶ **Overlapping of phases not supported:** Iterative waterfall model assumes that one phase can start after completion of the previous phase, But in real projects, phases may overlap to reduce the effort and time needed to complete the project.
- ▶ **Risk handling not supported:** Projects may suffer from various types of risks. But, Iterative waterfall model has no mechanism for risk handling.
- ▶ **Limited customer interactions:** Customer interaction occurs at the start of the project at the time of requirement gathering and at project completion at the time of software delivery. These fewer interactions with the customers may lead to many problems as the finally developed software may differ from the customers' actual requirements.

Evolutionary Process Models

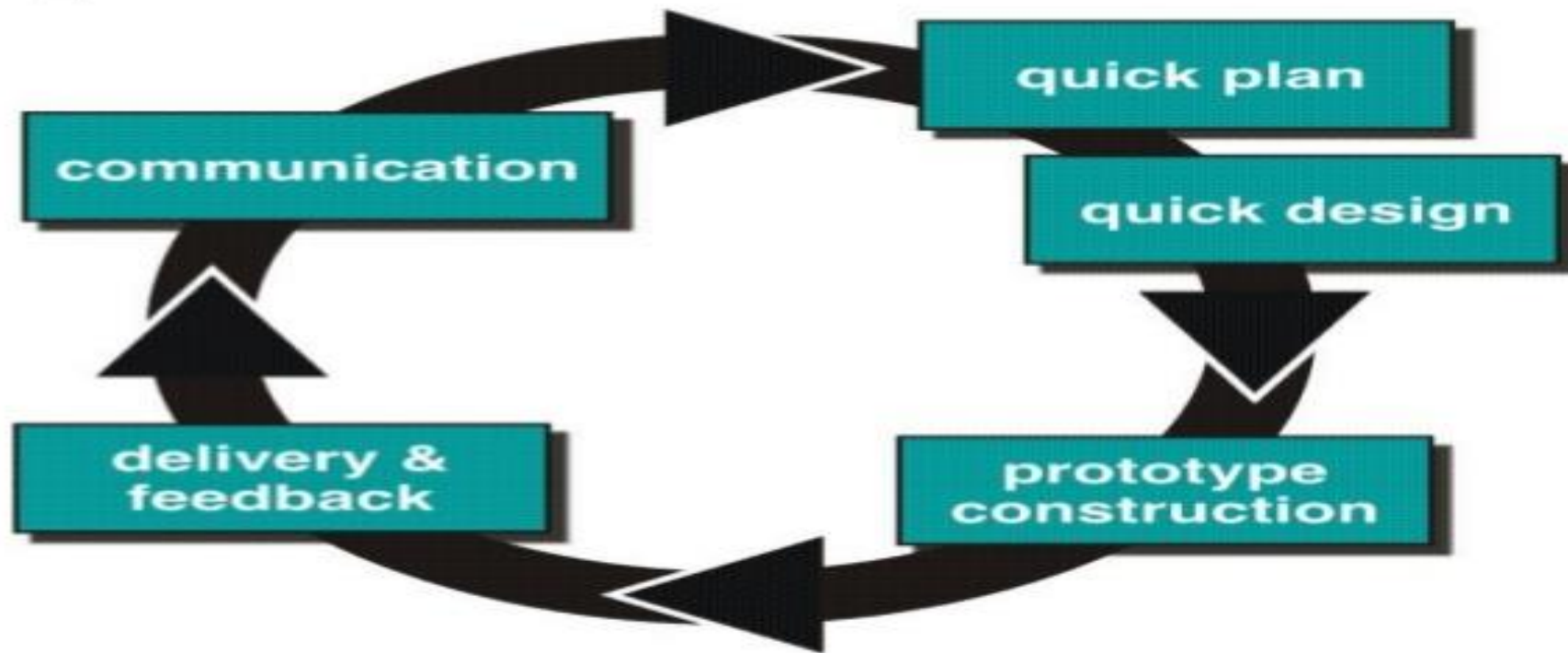


1. Prototyping Model

- Prototyping is defined as the process of developing a working replication of a product or system that has to be engineered.
- This model is used when the customers do not know the exact project requirements beforehand. In this model, a prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.
- In this process model, the system is partially implemented before or during the analysis phase thereby giving the customers an opportunity to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model. This document is used to build the initial prototype supporting only the basic functionality as desired by the customer. Once the customer figures out the problems, the prototype is further refined to eliminate them. The process continues till the user approves the prototype and finds the working model to be satisfactory.



[Evolutionary Process Models: Prototyping]



Evolutionary Process Model

2. Spiral Model

- Proposed by Boehm in 1988. Spiral model is one of the most important Software Development Life Cycle models, which provides support for **Risk Handling**. This model of development combines the features of the prototyping model and the waterfall model. The spiral model is favored for large, expensive, and complicated projects. The spiral model is similar to the incremental model, with more emphases placed on risk analysis.
- In its diagrammatic representation, it looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. **Each loop of the spiral is called a Phase of the software development process.** One phase is split roughly into four sectors of major activities: **Planning, Risk Analysis, Engineering or development and Evaluation.**
- Requirements are gathered during the **planning phase**. In the **risk analysis phase**, a process is undertaken to identify risk and alternate solutions. A prototype is produced at the end of the risk analysis phase.
- Software is produced in the **engineering phase**, along with testing at the end of the phase. The **evaluation phase** allows the customer to evaluate the output of the project to date before the project continues to the next spiral.

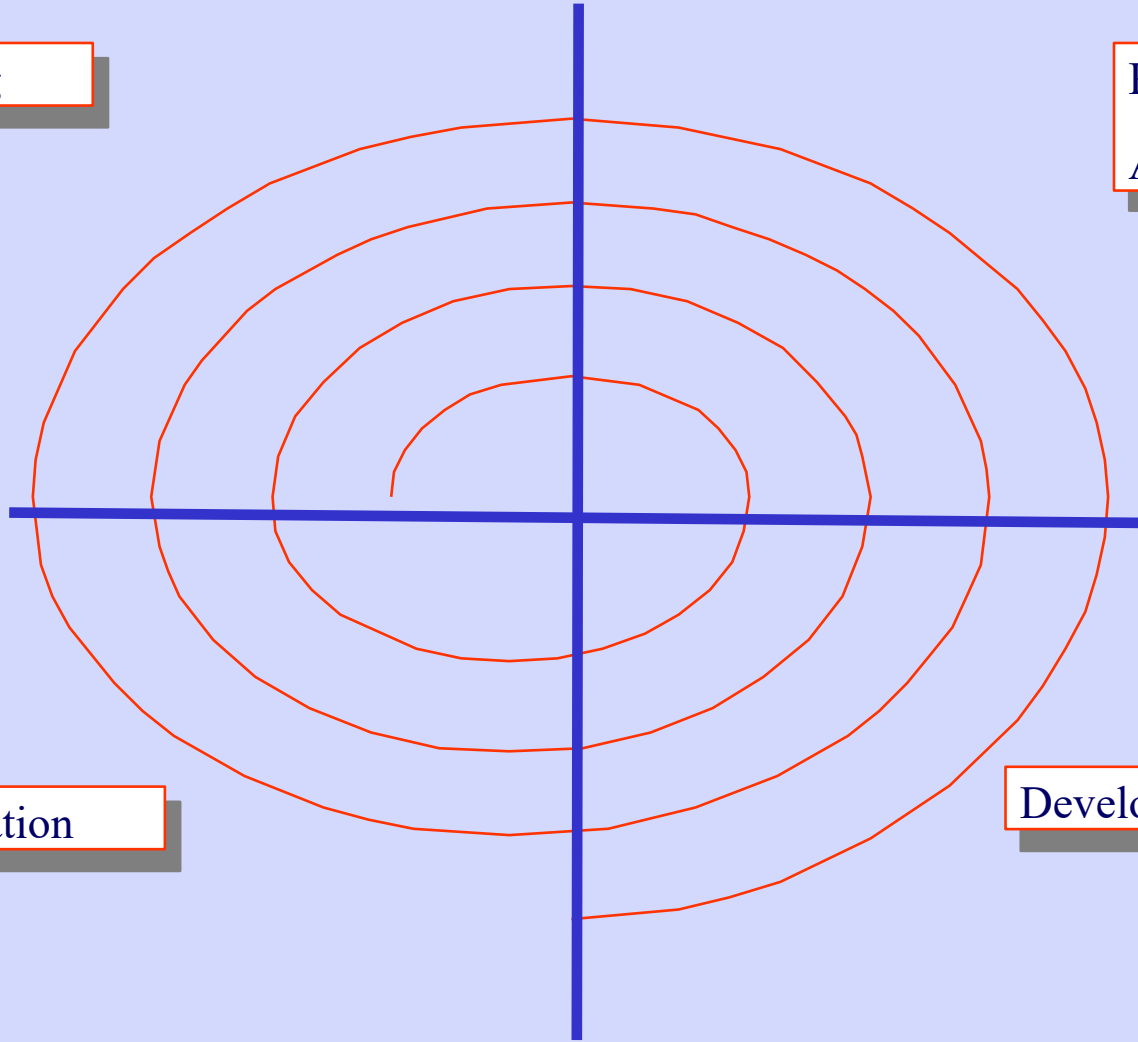
Planning

Risk
Analysis

Go/No-Go
Decision

Development

Evaluation





Planning: Determination of objectives, alternatives & constraints.

Risk Analysis: Analyze alternatives and attempts to identify and resolve the risks involved.

Development: Product development and testing product.

Evaluation: Customer evaluation.

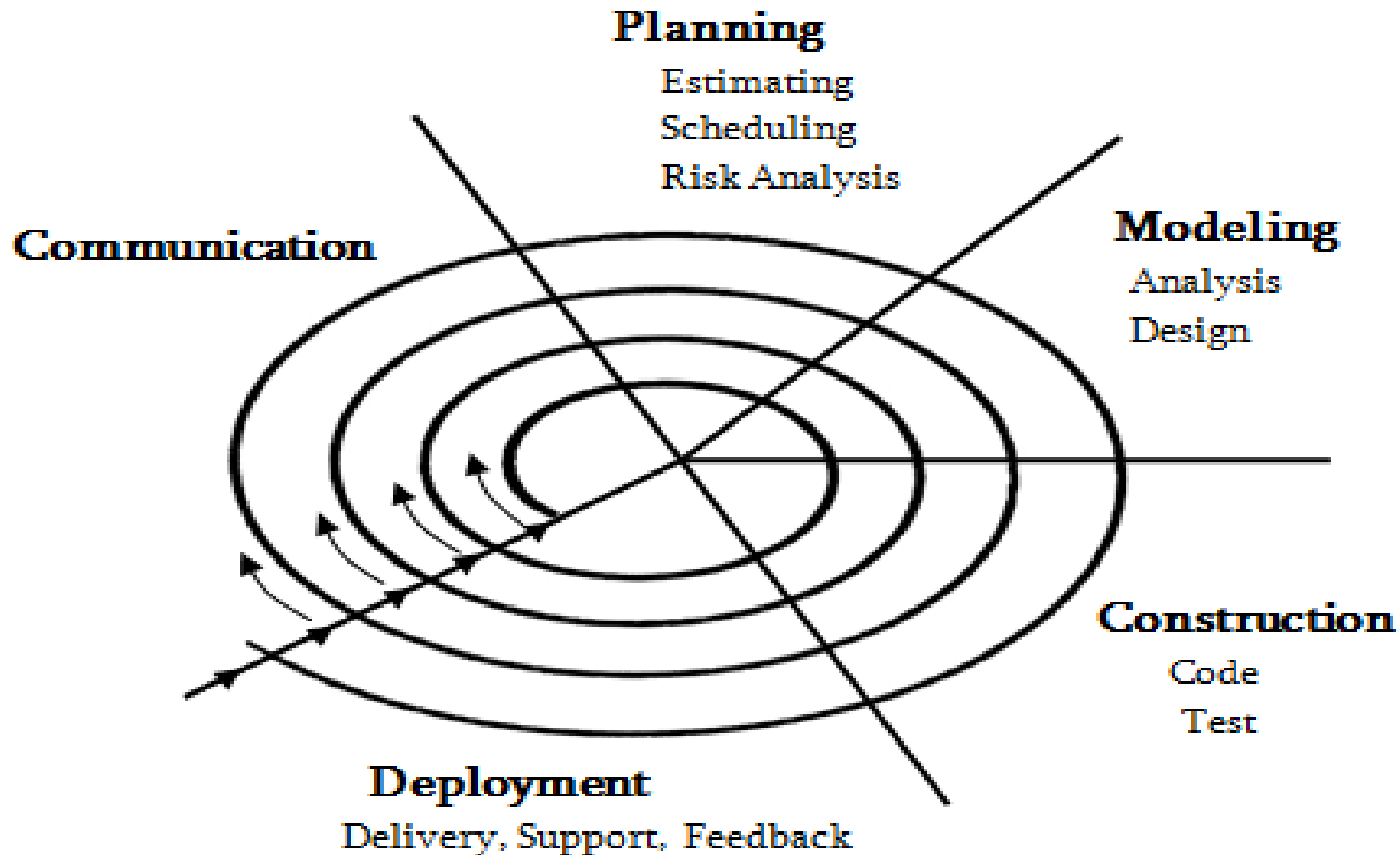


Figure : Spiral Model

Incremental Process Model:

1. Incremental Model: (Incremental process model is also known as Successive version model)

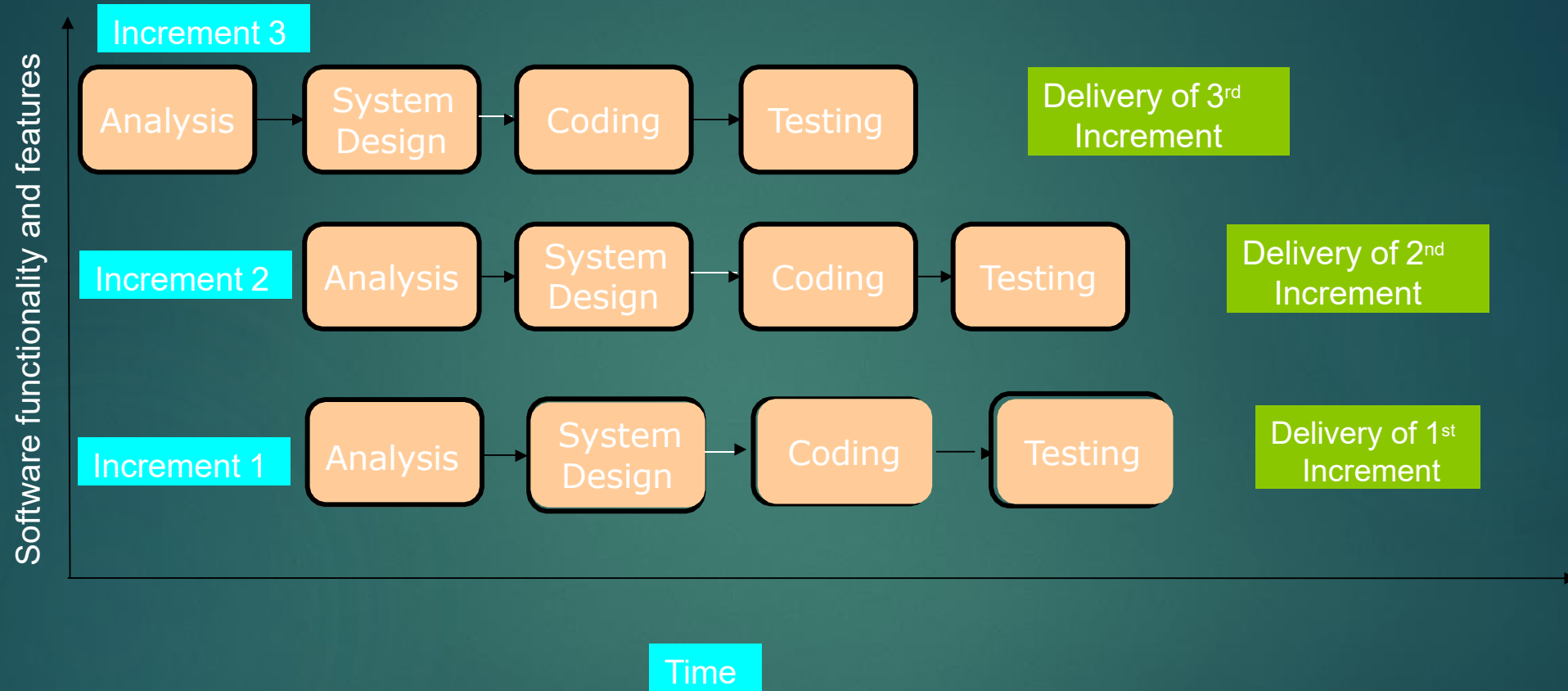
This model does not attempt to start with full specification of requirements. First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/ versions are implemented and delivered to the customer until the desired system is realized.

Multiple development cycles take place here, making the life cycle a “multi-waterfall” cycle. Each iteration passes through the requirements, design, implementation and testing phases.

Key Points

- Development and delivery is broken down into increments. Each increment delivers part of the required functionality.
- Requirements are prioritised and the highest priority requirements are included in early increments.
- Once the development of an increment is started, the requirements are frozen. Requirements for later increments can continue to evolve.

Incremental process model (contd.)

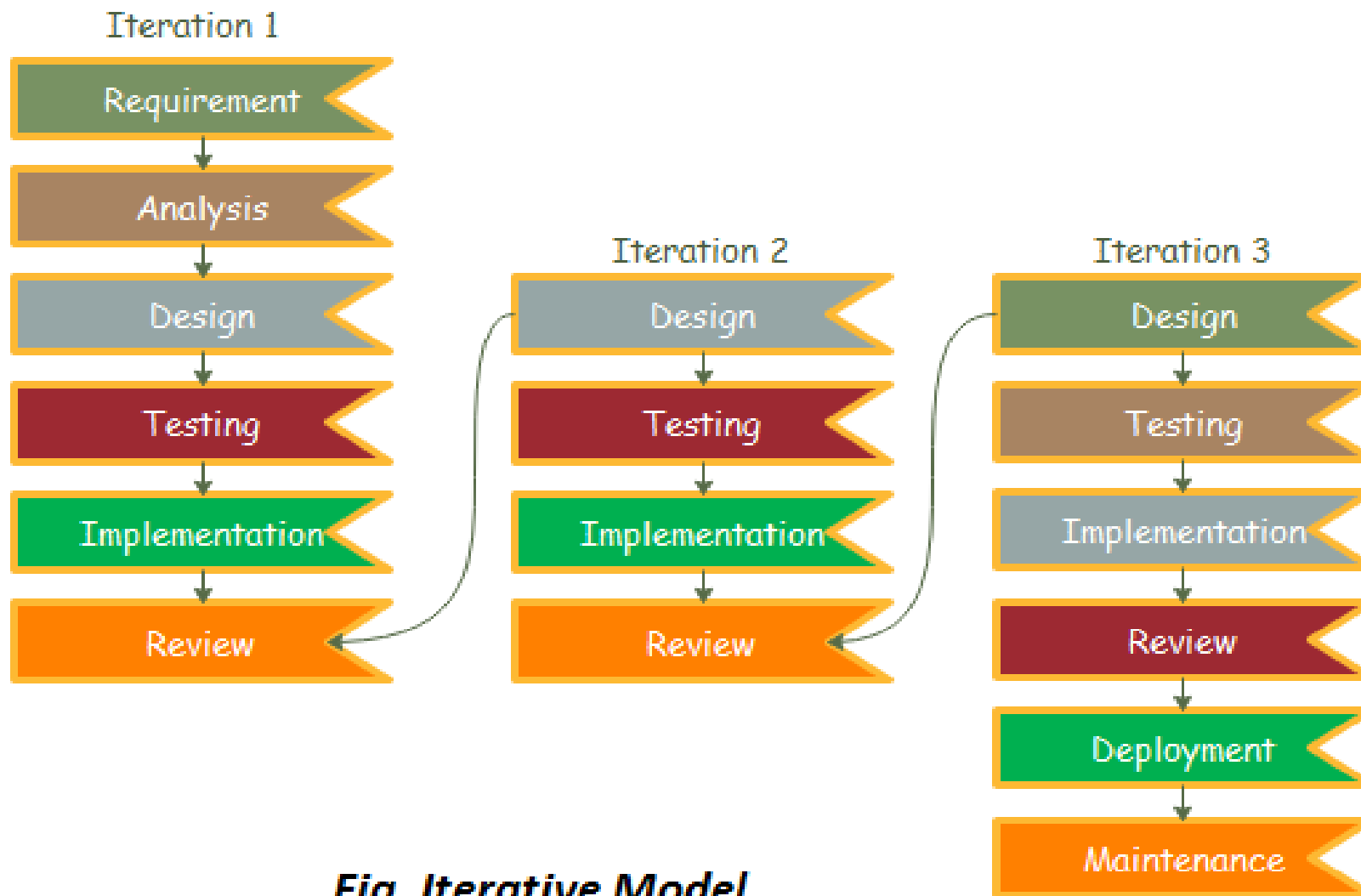


Iterative Model:

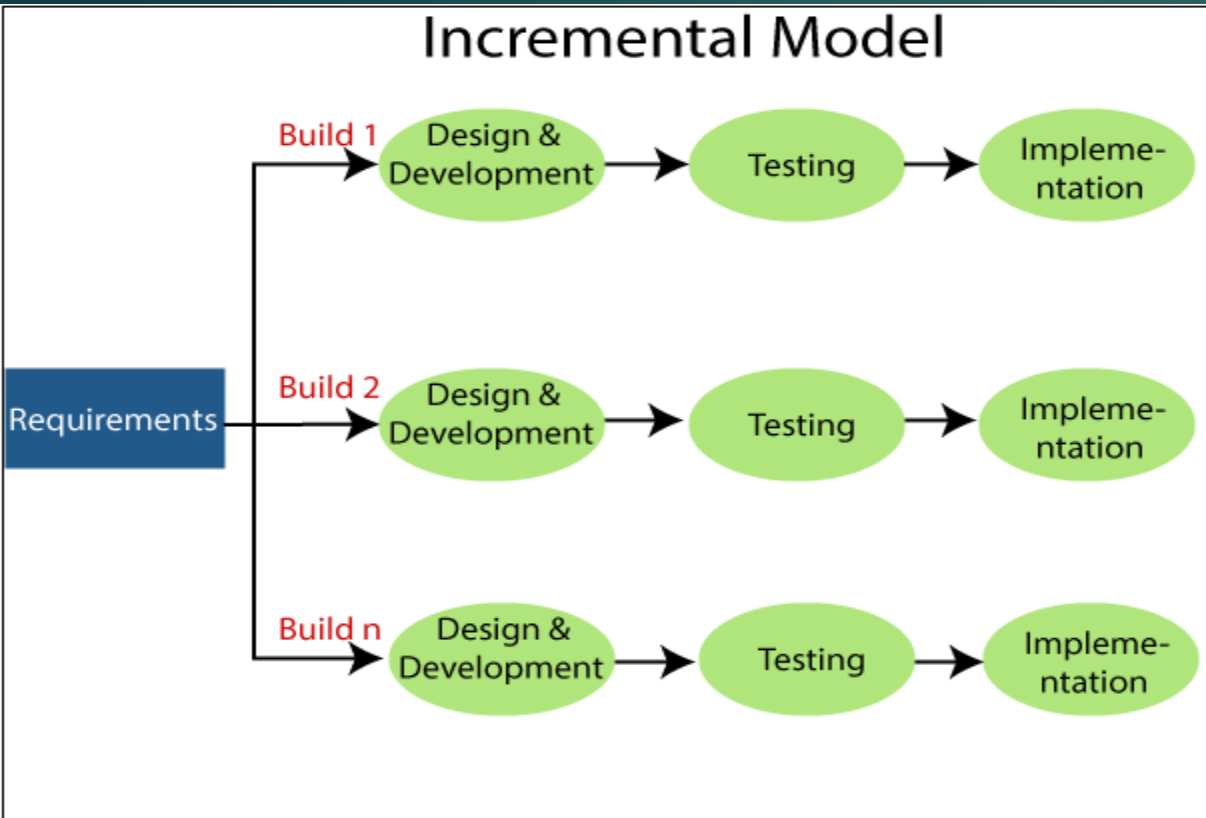
1. In this Model, you can start with some of the software specifications and develop the first version of the software. After the first version if there is a need to change the software, then a new version of the software is created with a new iteration. Every release of the Iterative Model finishes in an exact and fixed period that is called iteration.
2. The Iterative Model allows the accessing earlier phases, in which the variations made respectively. The final output of the project renewed at the end of the Software Development Life Cycle (SDLC) process.

When to use the Iterative Model?

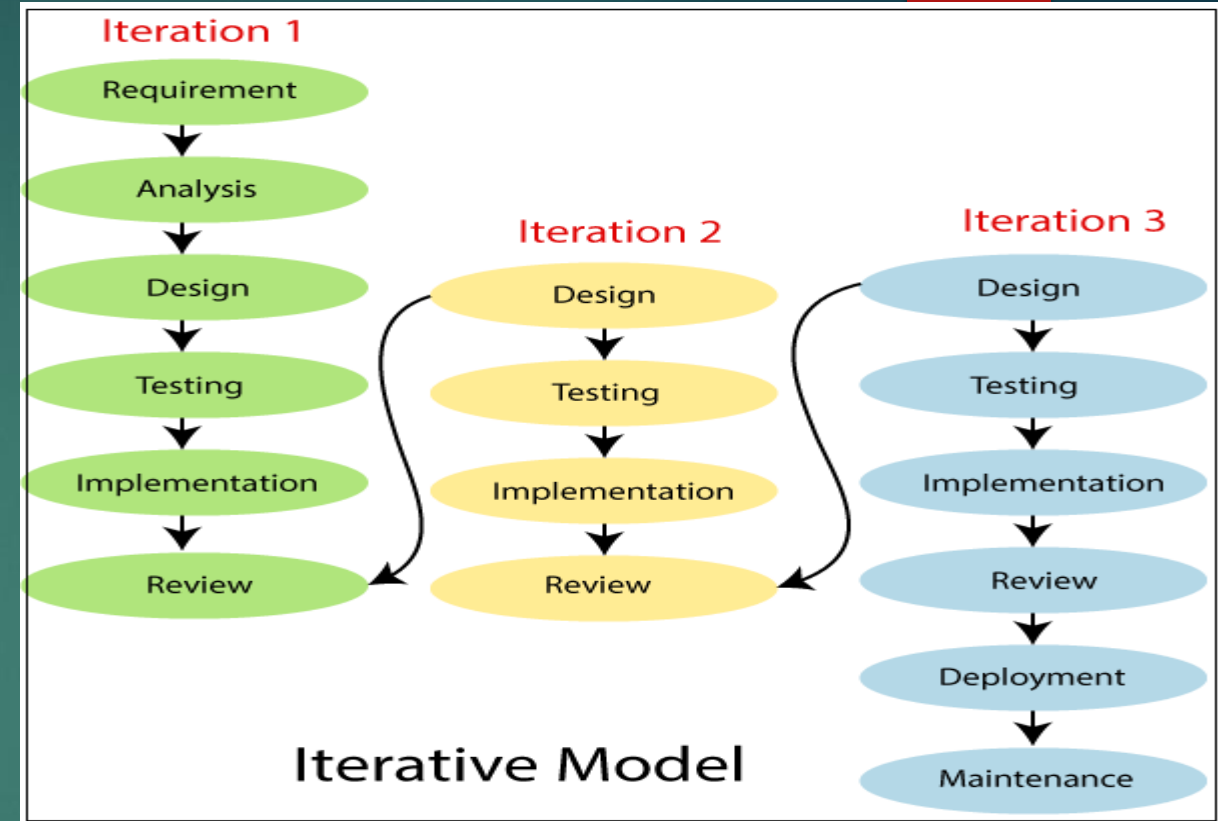
- When requirements are defined clearly and easy to understand.
- When the software application is large.
- When there is a requirement of changes in future.



Incremental Vs Iterative



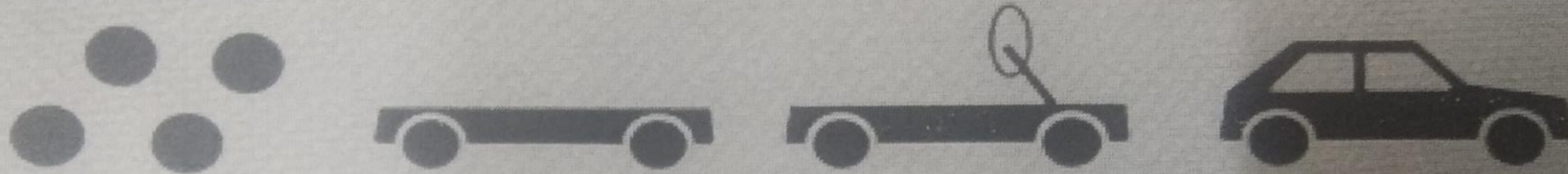
Incremental Model is a software development process where requirements are divided into several stand-alone software development modules. In this example, each module passes through the requirement, design, development, implementation, and testing phases. That subsequent release of the module adds a feature to the previous release. The process will continue until the whole software is achieved.



This model consists of the same phases as the waterfall model, but with fewer restrictions. Generally, the phases occur in the same order as in the waterfall model, but these may be conducted in several cycles. A reusable product is released at the end of the cycle, with each release providing additional functionality.

INCREMENTAL VS ITERATIVE

Incremental



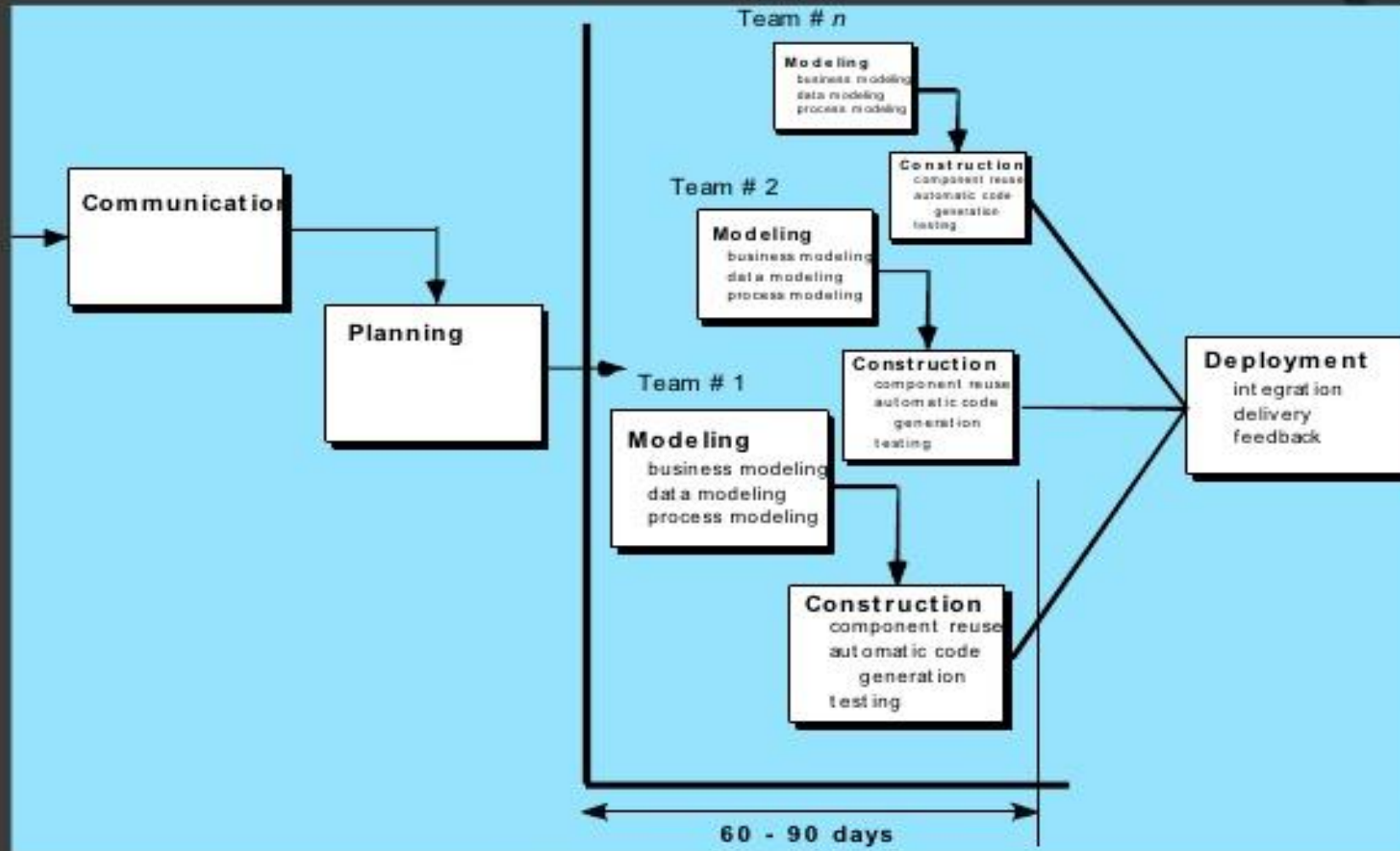
Iterative



Rapid Application Development (RAD) model

- The Rapid Application Development Model was first proposed by IBM in 1980's.
- A software project can be implemented using this model if the project can be broken down into small modules wherein each module can be assigned independently to separate teams. These modules can finally be combined to form the final product.
- Development of each module involves the various basic steps as in waterfall model i.e analyzing, designing, coding and then testing, etc. as shown in the figure.

RAD Model



conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Specialized Process Models

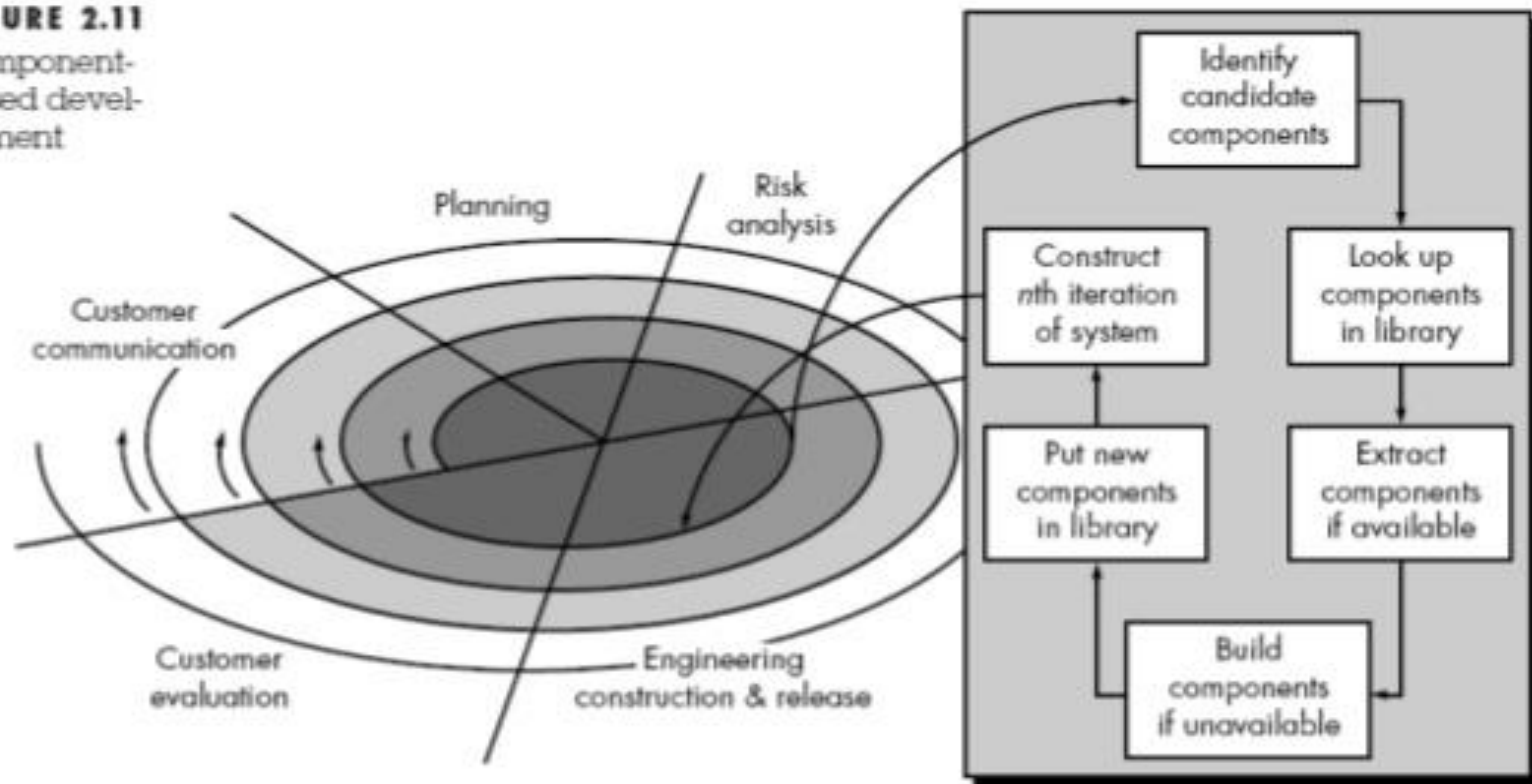
1. Component Based Development Model

- Building software from reusable components.
- The CBD model leads to software reuse, and reusability provides software engineers with a number of measurable benefits.
- Object oriented technologies provide the technical framework for a component based process model. The CBD model composes application from prepackaged software components (called classes). Once candidate classes are identified, the class library is searched to determine if these classes already exist. If they do, they are extracted from the library and reused. If a candidate class does not reside in the library, it is engineered using object oriented methods.

[Component Based Development Model]

FIGURE 2.11

Component-based development



V - model

- The Verification and Validation model commonly known as V Model is considered to be an extension of the waterfall model.
- The V-model is a type of SDLC model where process executes in a sequential manner in V-shape. It is based on the association of a testing phase for each corresponding development stage. Development of each step directly associated with the testing phase. The next phase starts only after completion of the previous phase i.e. for each development activity, there is a testing activity corresponding to it.
- V-Model contains Verification phases on one side of the Validation phases on the other side. Verification and Validation phase joined by coding phase in V-shape. So it is called as V-Model.

Note:

Verification: It is the process of evaluation of the product development phase to find whether specified requirements meet. Verification is static testing.

Verification means **Are we building the product right?**

Validation: It involves dynamic analysis technique (functional, non-functional), testing done by executing code. Validation is the process to evaluate the software after the completion of the development phase to determine whether software meets the customer expectations and requirements. Validation is the dynamic testing.

Validation means **Are we building the right product?**



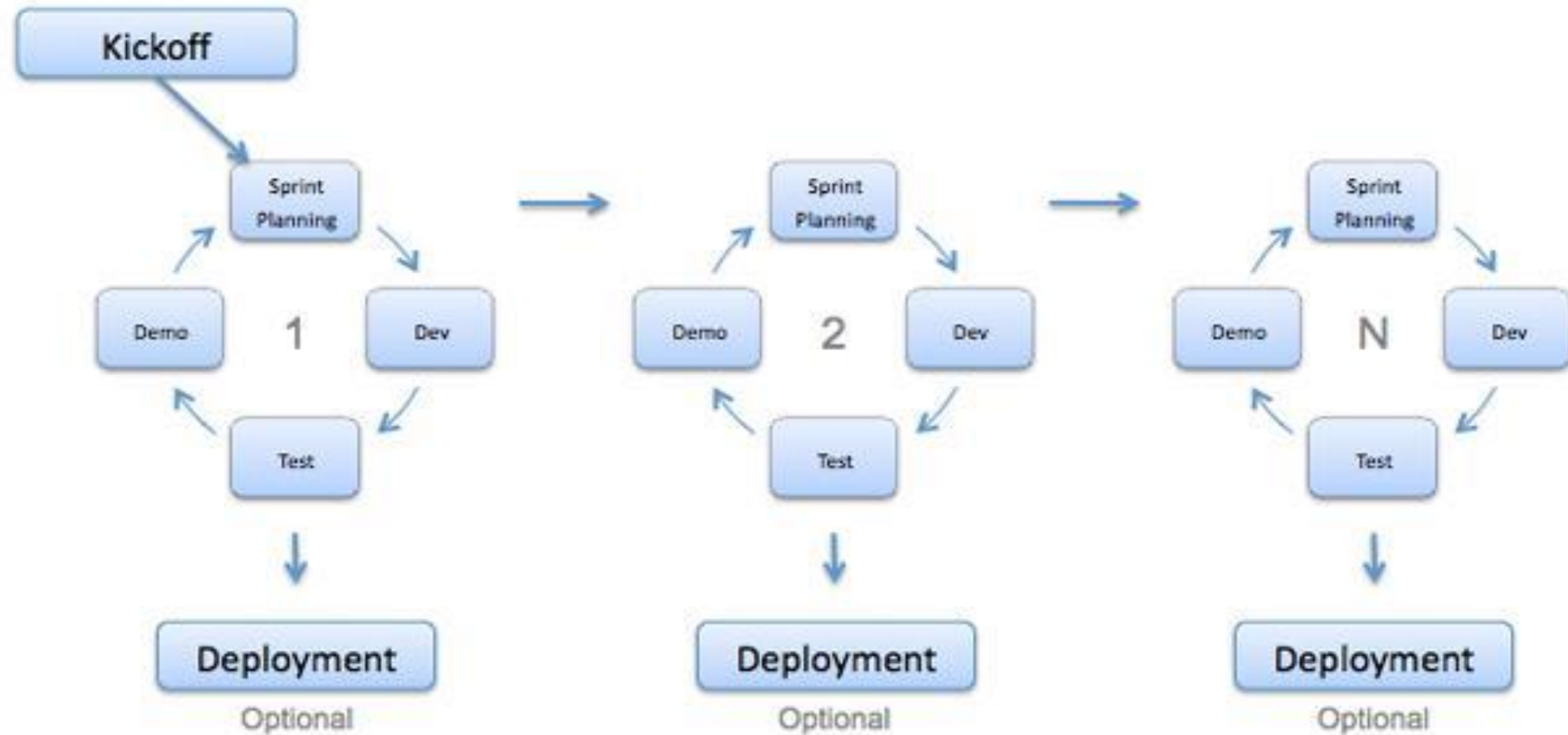
Agile Process Models:

- The Agile model was primarily designed to help a project to adapt to change requests quickly. So, the main aim of the Agile model is to facilitate quick project completion. To accomplish this task agility is required. Agile means to be swift or versatile and refers **to rapid reaction to changes in framework conditions**.
- Agile method proposes incremental and iterative approach to software design.
- The customer has early and frequent opportunities to look at the product and make decision and changes to the project.
- Development process is iterative, and the project is executed in short (2-4) weeks iterations. Rapid, incremental delivery of software. Planning is very less.
- Customers, developers and testers constantly interact with each other.
- At the end of every sprint, user acceptance is performed.

A few Agile SDLC models are given below:

- Extreme programming (XP)
- Adaptive Software Development (ASD)
- Scrum
- Crystal

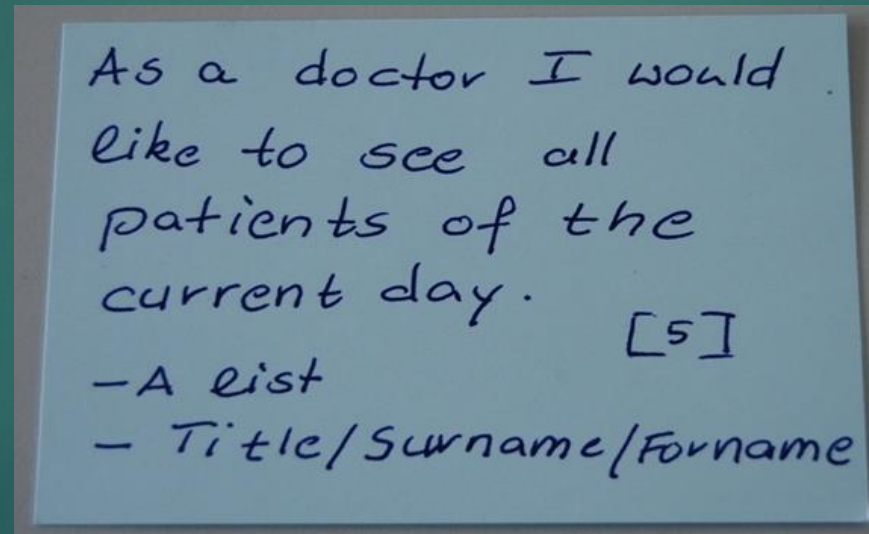
Agile Model:



Agile Process Models:

1. Extreme programming (XP)

- XP is based on the frequent iteration through which the developers implement User Stories. User stories are simple and informal statements of the customer about the functionalities needed.
- Requirements in XP are captured in the form of user stories on story cards. Each story (similar to use cases) is written by the customer and is placed on an index card. Diagram shows Story card as an index card.



As a doctor I would
like to see all
patients of the
current day. [5]
- A list
- Title/Surname/Forname

- On the basis of User stories, the development team may decide to build a Spike for some feature. A Spike is a very simple program that is constructed to explore the suitability of a solution being proposed. It can be considered similar to a prototype.

- ▶ Use an object-oriented approach
- ▶ XP Planning
 - ▶ Begins with the creation of “user stories”, i.e., use-cases
 - ▶ Agile team assesses each story and assigns a cost
 - ▶ Stories are grouped to for a deliverable increment
 - ▶ A commitment is made on delivery date
 - ▶ After the first increment “project velocity” is used to help define subsequent delivery dates for other increments
- ▶ XP Design
 - ▶ Follows the KIS (keep it simple) principle
 - ▶ Encourage the use of CRC (Class-Responsibility Collaborator) cards
 - ▶ For difficult design problems, suggests the creation of “spike solutions” —a design prototype
 - ▶ Encourages “refactoring” —an iterative refinement of the internal program design

**number of stories
implemented during first
release**

▶ XP Coding

- ▶ Recommends the construction of a unit test for a story *before* coding commences
- ▶ Encourages “pair programming”

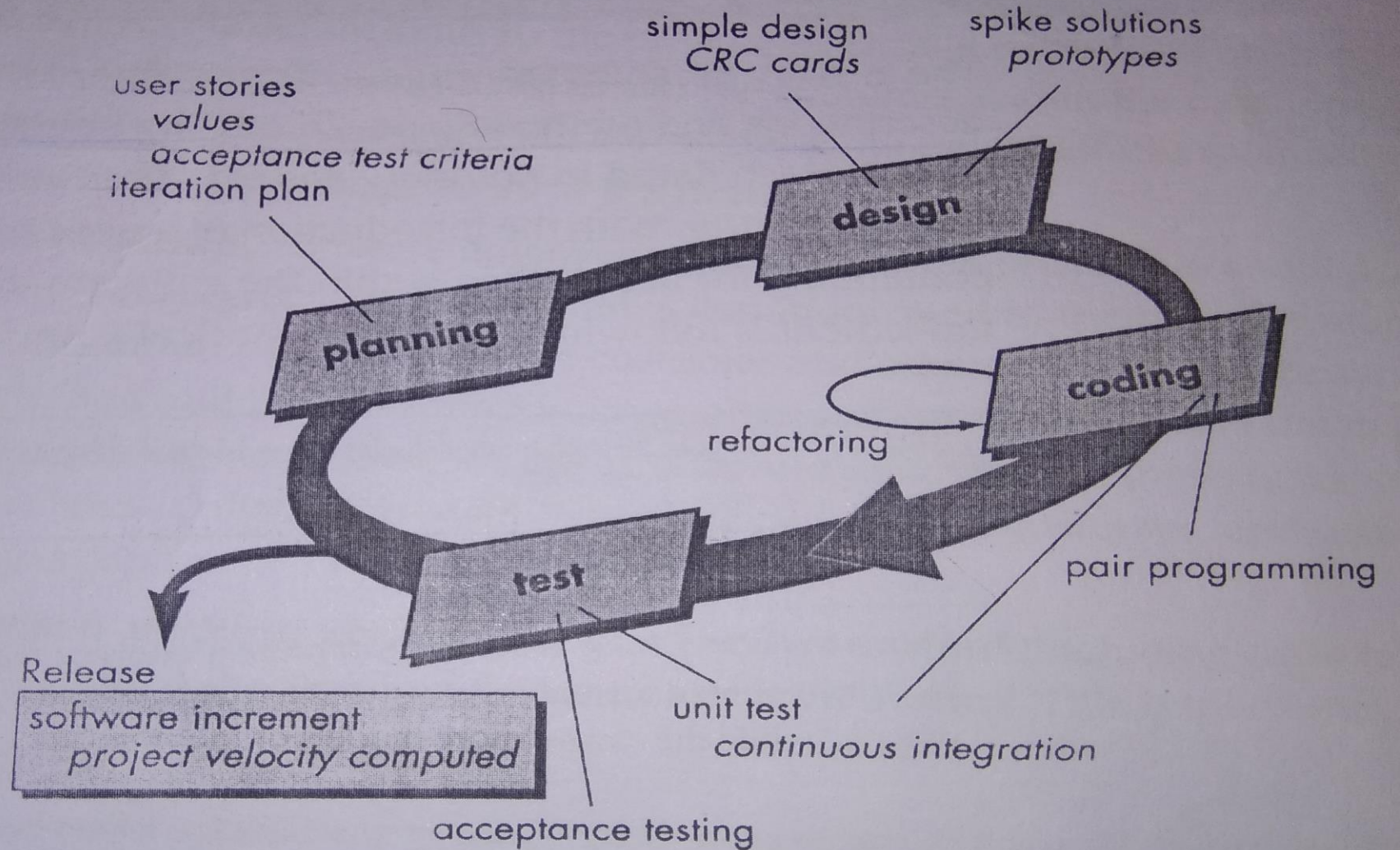
**Two programmers
work together for
coding a story**

▶ XP Testing

- ▶ All unit tests are executed daily
- ▶ “Acceptance tests” are defined by the customer and executed to assess customer visible functionality

Agile Process Models:

The Extreme Programming process



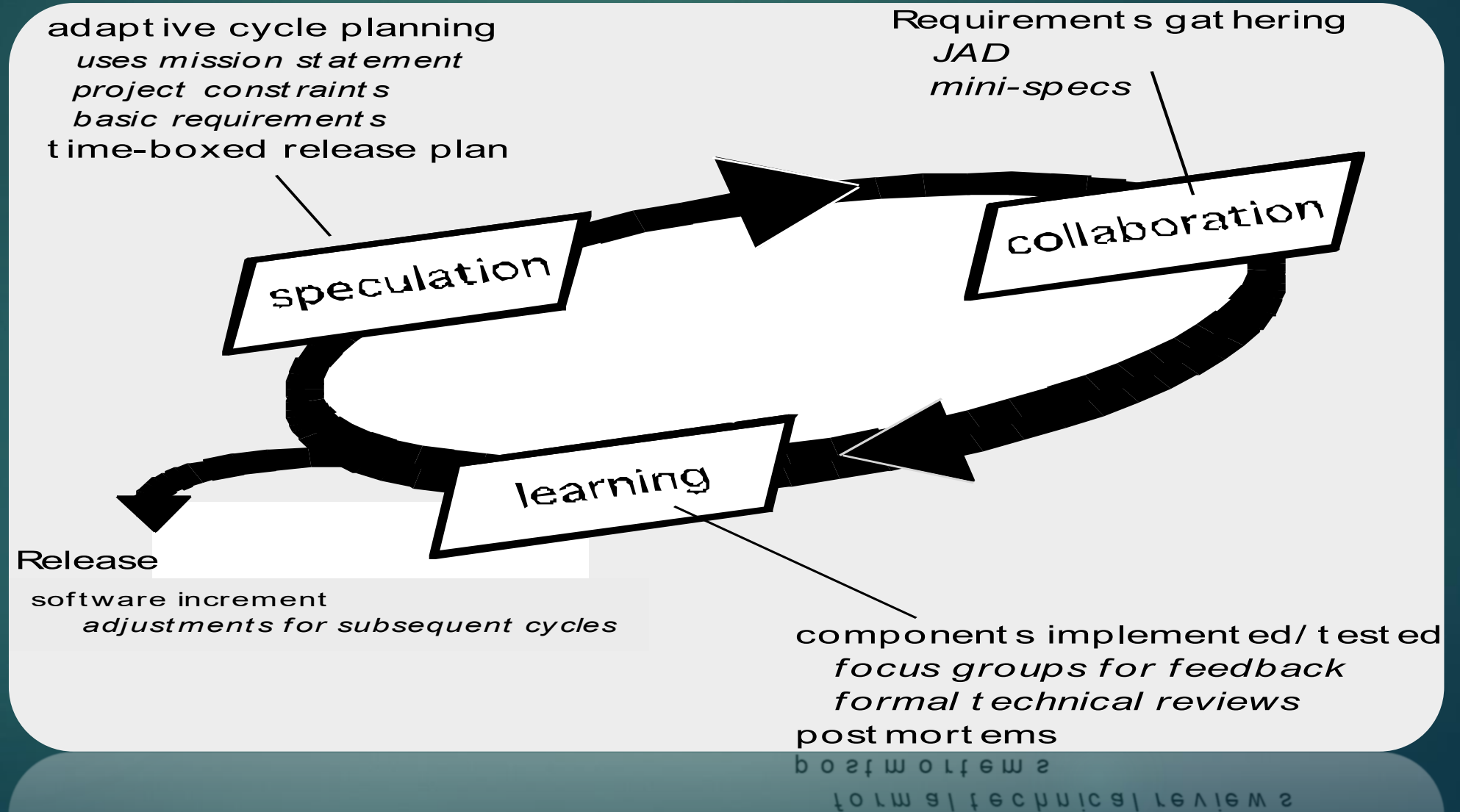
Agile Process Models:

2. Adaptive Software Development: ASD focus on human collaboration and team self organization.

- ▶ Originally proposed by Jim Highsmith
- ▶ ASD — distinguishing features
 - ▶ Mission-driven planning
 - ▶ Component-based focus
 - ▶ Uses “time-boxing”. In time management, a time box allots a fixed period of time for an activity
 - ▶ Explicit consideration of risks
 - ▶ Emphasizes collaboration for requirements gathering
 - ▶ Emphasizes “learning” throughout the process

Adaptive Software Development

50



ASD life cycle that incorporate three phases:

Speculation: During speculation, the project is initiated and adaptive cycle planning is conducted. Adaptive cycle planning uses project initiation information (the customer's mission statement, project constraints e.g.. Delivery dates etc.), and basic requirements.

Collaboration: Motivated people work together in a way that multiplies their talent and creative output beyond their absolute numbers.

Learning: As member of an ASD team begin to develop the components that are part of an adaptive cycle, the emphasis is on learning as much as it is on progress toward a completed cycle.

3. Scrum: SCRUM is an agile development method which concentrates specifically on how to manage tasks within a team-based development environment.

Originally proposed by Jeff Sutherland et al. in the early 1990s.

Scrum—distinguishing features

- Development work is partitioned into “packets”
- Testing and documentation are on-going as the product is constructed
- Work occurs in “sprints” and is derived from a “backlog” of existing requirements
- Meetings are very short and sometimes conducted without chairs
- “demos” are delivered to the customer with the time-box allocated

Scrum principles are consistent with the agile manifesto:

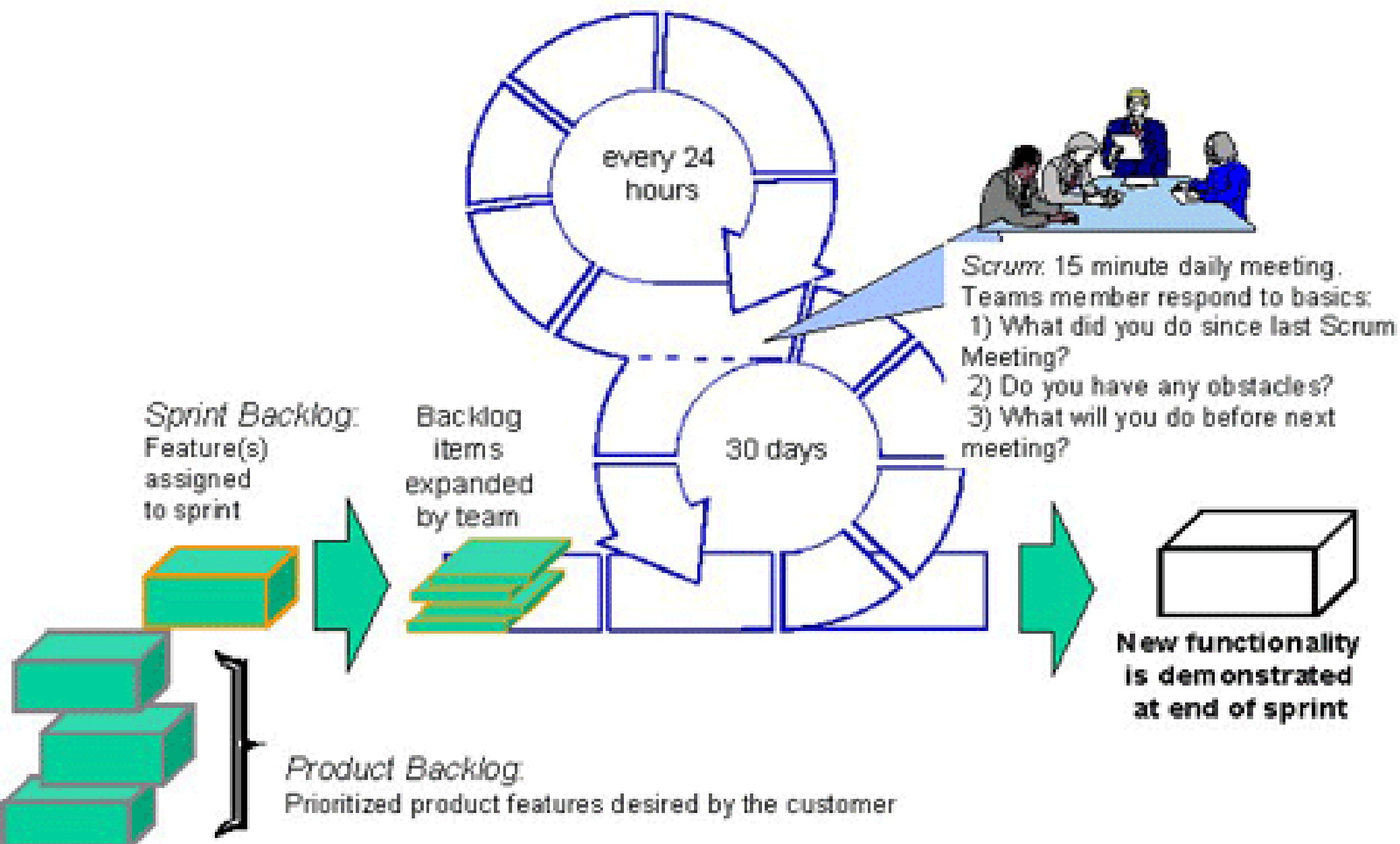
- Small working teams are organized to “maximize communication, minimize overhead, and sharing of tacit, informal knowledge.
- The process must be adaptable to both technical and business changes “to ensure the best possible product is produced”.
- The process yields frequent software increments “that can be inspected, adjusted, tested, documented, and built on.”
- Development work and the people who perform it are partitioned “into clean low coupling partitions or packets”.
- Constant testing and documentation is performed as product is built.

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

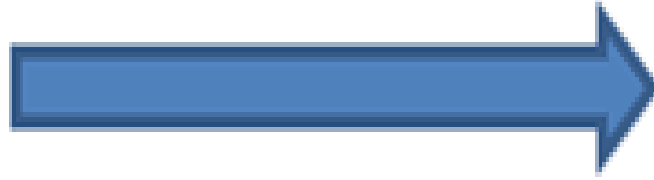
Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.





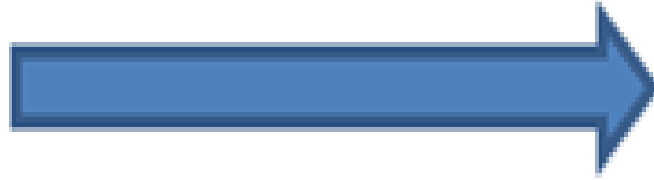
Scrum Master



Scrum Meeting



Product Owner



Prepare product backlog



Team organizes tasks

Scrum Master

- Master is responsible for setting up the team, sprint meeting and removes obstacles to progress

Product owner

- The Product Owner (PO) is a member of the Agile Team responsible for defining Stories and prioritizing the Team Backlog to streamline the execution of program priorities.
- The Product Owner creates product backlog, prioritizes the backlog and is responsible for the delivery of the functionality at each iteration

Scrum Team

- Team manages its own work and organizes the work to complete the sprint or cycle

Product Backlog

- This is a repository where requirements are tracked with details on the no of requirements to be completed for each release. It should be maintained and prioritized by Product Owner, and it should be distributed to the scrum team. Team can also request for a new requirement addition or modification or deletion.

Sprint:

- Each iteration of a scrum is known as Sprint. Sprint consist of work units that are required to achieve a requirement defines in the backlog that must be fit into a predefined time box (typically 30 days). At the end of the sprint, team delivers product functionality.

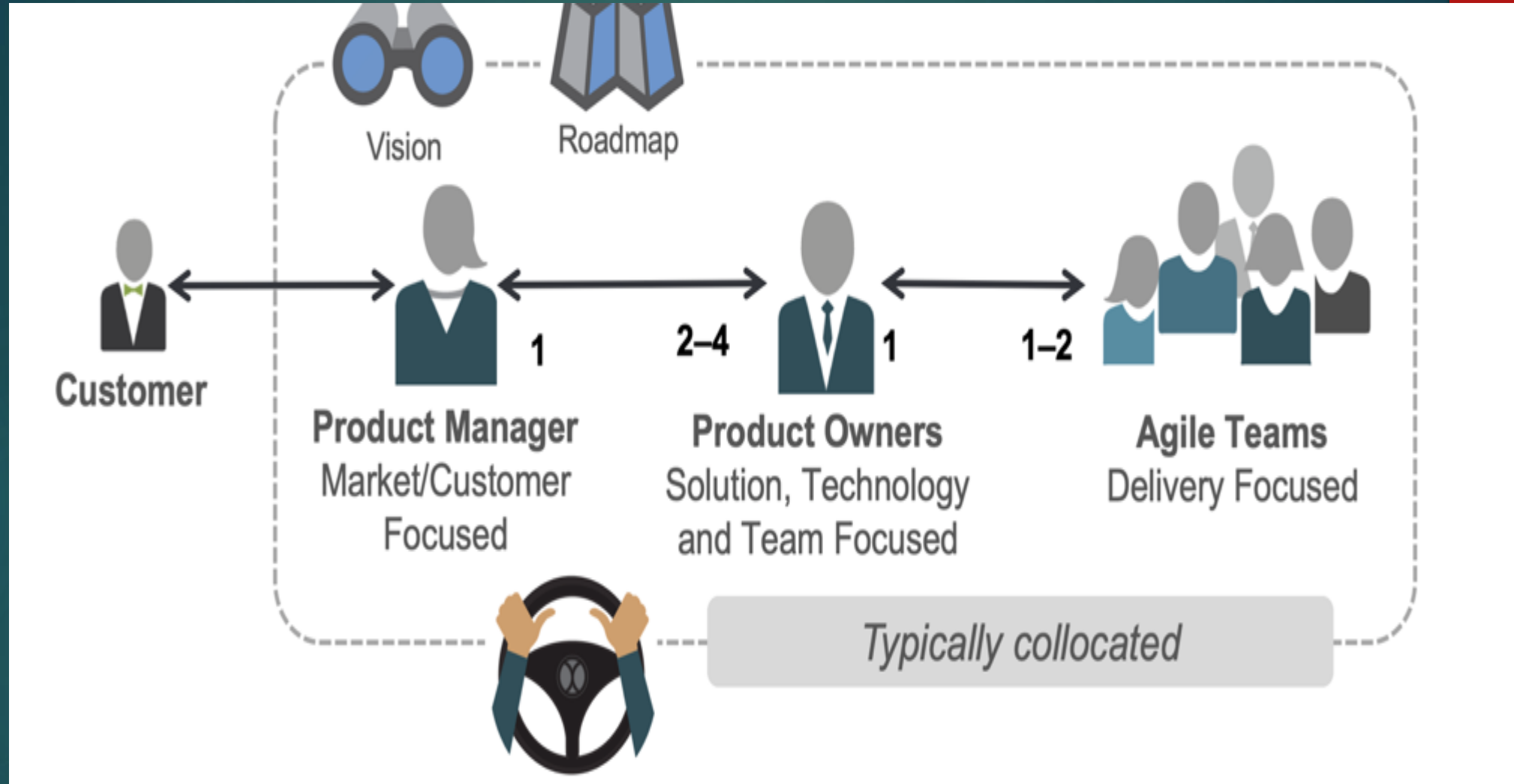


Figure: Fan-out model for Product Manager, PO, and Agile teams

4. Crystal: The Crystal methodology is one of the most lightweight, adaptable approaches to software development.

► Proposed by Cockburn and Highsmith

► Crystal—distinguishing features

- The methods are color-coded to signify the risk to human life. For example, projects that may involve risk to human life will use Crystal Sapphire while projects that do not have such risks will use Crystal Clear.
- Crystal focuses on six primary aspects: people, interaction, community, communication, skills, and talents. Process is considered secondary. There are also seven common properties in Crystal that indicate higher possibility of success and they include frequent delivery, reflective improvement, osmotic communication, and easy access to expert users.
- Crystal Methods consider people as the most important, so processes should be modeled to meet the requirements of the team. It is adaptive, without a set of prescribed tools and techniques. It is also lightweight, without too much documentation, management or reporting. For example, Crystal Clear is for short-term projects by a team of 6 developers working out of a single workspace.

The 7 Properties of Crystal Clear

